Visualise cophylogenetic signals

Phylogeny x Bipartite Network Workflows

Table of contents

はじめに 5
収録チャプター 5

第 1 章 ggbipartite で始める二部ネットワーク可視化 7
1.1 事前準備 7
1.2 まず系統樹を可視化して全体像を掴む 7
1.3 相互作用行列を作り、系統樹の順序に合わせる 9
1.4 long 形式へ変換し `row`, `column`, `count` を理解する 10
1.5 abundance の基本可視化 11
1.6 メタデータで色分けする 13
1.7 レイアウトパラメータを調整する 15
1.8 binary（有無）表示に切り替える 17
1.9 `construct_bn_coordination()` でラベル注釈する 18
1.10 系統樹とネットワークを対応づける 20
1.11 応用編: 系統樹スケールを維持した可視化 25
1.12 よくあるつまずき 28
1.13 まとめ 29

第 2 章 ggbipartite で作る Publishable な系統樹 × 二者ネットワーク図 31
2.1 事前準備 31
2.2 データと系統樹を作る 32
2.3 系統樹を注釈する 34
2.4 marquee で tip/node ラベルの見た目を揃える 36
2.5 abundance 版の図を作る 36
2.6 binary 版の図を作る 41
2.7 binary + RPANDA シグナルを重ねる（`geom_nodepoint()`） 46
2.8 出力時の実務メモ 50

第 3 章 学名付きの系統樹ラベルを整える 51
3.1 事前準備 51

3.2 ダミー系統樹とラベルを用意する 51
3.3 学名とサポート値を整形する 52
3.4 align = FALSE: tip ごとにラベルを寄せる 54
3.5 align = TRUE: 補助線を付けて整列する 55
3.6 tip ラベルの見切れを防ぐ（`expand = expansion(mult = ...)`） . . . 56
3.7 `scale_x_reverse()` で左右反転する 57
3.8 仕様解説 59
3.8.1 `expand = expansion(mult = c(0.70, 0.02))` 59
3.8.2 `format_node_support()` の仕様 59
3.8.3 `style_tree_label()` の仕様 60
3.8.4 `style_sciname()` の仕様 61
3.9 実務メモ 62

第 4 章 Session info 63

はじめに

収録チャプター

本編は次の 4 章で進みます。

1. `ggbipartite` で始める二部ネットワーク可視化
2. `ggbipartite` で作る Publishable な系統樹 × 二者ネットワーク図
3. 学名付きの系統樹ラベルを整える
4. Session info（実行環境）

```
# pak::pak("Hide-Fun/ggbipartite")
packageVersion("ggbipartite")
```

```
[1] '0.9.5'
```

第 1 章

ggbipartite で始める二部ネットワーク可視化

この vignette は、`ggbipartite` を使った二部ネットワーク可視化を初心者向けに段階的に解説します。

最終的に、次の 4 パターンを自分のデータで再現できる状態を目指します。

1. abundance（量）を反映した基本可視化
2. メタデータに基づく色分け
3. binary（有無）に特化した可視化
4. 系統樹とネットワークの対応づけ

1.1 事前準備

```
library(ggplot2)
library(dplyr)
library(tidyr)
library(tibble)
library(ape)
library(ggtree)
library(ggbipartite)
library(patchwork)
```

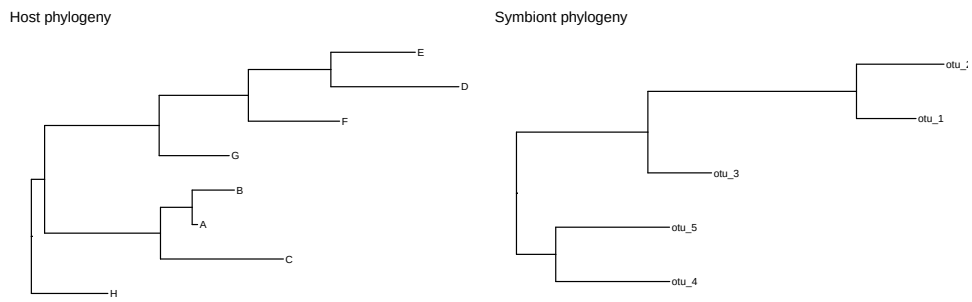
1.2 まず系統樹を可視化して全体像を掴む

この章で使う主な関数:

- `rtree()`: 例示用のランダム系統樹を生成します。
- `ggtree()`: 系統樹を `ggplot` オブジェクトとして描画します。
- `geom_tiplab()`: tip ラベルを表示します。
- `patchwork::plot_layout()`: 2 つの tree を横並びで表示します。

最初に `host_phylo` と `symbiont_phylo` を単体で描くと、「左右どちらがどの生物群か」をイメージしやすくなります。

```
set.seed(123)

host_phylo <- rtree(8)
host_phylo$tip.label <- LETTERS[1:8]

symbiont_phylo <- rtree(5)
symbiont_phylo$tip.label <- sprintf("otu_%d", 1:5)

p_host_tree <- ggtree(host_phylo) +
  geom_tiplab(size = 3) +
  labs(title = "Host phylogeny")

p_symbiont_tree <- ggtree(symbiont_phylo) +
  geom_tiplab(size = 3) +
  labs(title = "Symbiont phylogeny")

if (requireNamespace("patchwork", quietly = TRUE)) {
  p_host_tree +
    p_symbiont_tree +
    patchwork::plot_layout(nrow = 1)
} else {
  p_host_tree
  p_symbiont_tree
}
```



1.3　相互作用行列を作り、系統樹の順序に合わせる

この章で使う主な関数:

- `get_tip_order()`: tree 上の y 順に tip ラベル順を取得します。
- `base::factor(levels = ...)`: host / symbiont の表示順を固定します。
- `arrange()` と `relocate()`: 行列化前の表を順序どおりに並べます。
- `column_to_rownames()`: host 列を行名へ変換して行列化します。

`ggbipartite` では、行列の行順・列順がレイアウトに直接影響します。
そのため、系統樹と対応させる場合は以下の 2 点が重要です。

1. `factor(levels = ...)` で表示順を明示する
2. 行列をその順序でソートしてから可視化に使う

```
host_order <- get_tip_order(host_phylo) |> rev()
symbiont_order <- get_tip_order(symbiont_phylo) |> rev()

interaction_tbl <- tibble(
  host = LETTERS[1:8],
  otu_1 = c(40, 0, 20, 0, 5, 0, 8, 1),
  otu_2 = c(0, 9, 1, 4, 0, 2, 0, 0),
  otu_3 = c(10, 0, 0, 3, 4, 0, 1, 7),
  otu_4 = c(1, 0, 12, 0, 0, 6, 0, 0),
  otu_5 = c(0, 2, 0, 1, 8, 0, 0, 3)
)

interaction_matrix <- interaction_tbl |>
  arrange(factor(host, levels = host_order)) |>
  relocate(host, all_of(symbiont_order)) |>
  column_to_rownames(var = "host") |>
```

```
  as.matrix()

metadata_row <- tibble(
  host = rownames(interaction_matrix),
  family = c(
    "Fagaceae", "Fagaceae", "Rosaceae", "Rosaceae",
    "Fabaceae", "Fabaceae", "Betulaceae", "Betulaceae"
  )
)

metadata_column <- tibble(
  symbiont = colnames(interaction_matrix),
  guild = c("AMF", "AMF", "ECM", "ECM", "Unknown")
)

interaction_matrix
```

```
  otu_2 otu_1 otu_3 otu_5 otu_4
E     0     5     4     8     0
D     4     0     3     1     0
F     2     0     0     0     6
G     0     8     1     0     0
B     9     0     0     2     0
A     0    40    10     0     1
C     1    20     0     0    12
H     0     1     7     3     0
```

1.4　long 形式へ変換し `row`, `column`, `count` を理解する

この章で使う主な関数:

- `to_longer()`: 行列を long 形式へ変換します（0 は除外）。
- `rename()` と `mutate()`: `count` 列名と因子順を整えます。
- `ggbipartite` の主要 geoms は `row`, `column`, `count` を必須入力として使います。

`ggplot()` に渡すときは、最低限 `row`, `column`, `count` を使います。

1. `row`: 左側ノード ID（この vignette では host）
2. `column`: 右側ノード ID（この vignette では symbiont/otu）
3. `count`: 相互作用の強さ（カウント、重み）

```
interaction_df <- to_longer(interaction_matrix, .rowname = "row") |>
  rename(count = interaction) |>
  mutate(
    row = factor(row, levels = rownames(interaction_matrix)),
    column = factor(column, levels = colnames(interaction_matrix))
  )

glimpse(interaction_df)
```

```
Rows: 21
Columns: 3
$ row    <fct> E, E, E, D, D, D, F, F, G, G, B, B, A, A, A, C, C, C, H, H, H
$ column <fct> otu_1, otu_3, otu_5, otu_2, otu_3, otu_5, otu_2, otu_4, otu_1, ~
$ count  <dbl> 5, 4, 8, 4, 3, 1, 2, 6, 8, 1, 9, 2, 40, 10, 1, 1, 20, 12, 1, 7,~
```

原理的には、行列と系統樹の左右対応が保たれていれば row / column を逆に定義しても描画は可能です。
ただし実務では混乱を避けるため、最初に「row をどちら側とみなすか」を固定することを推奨します。

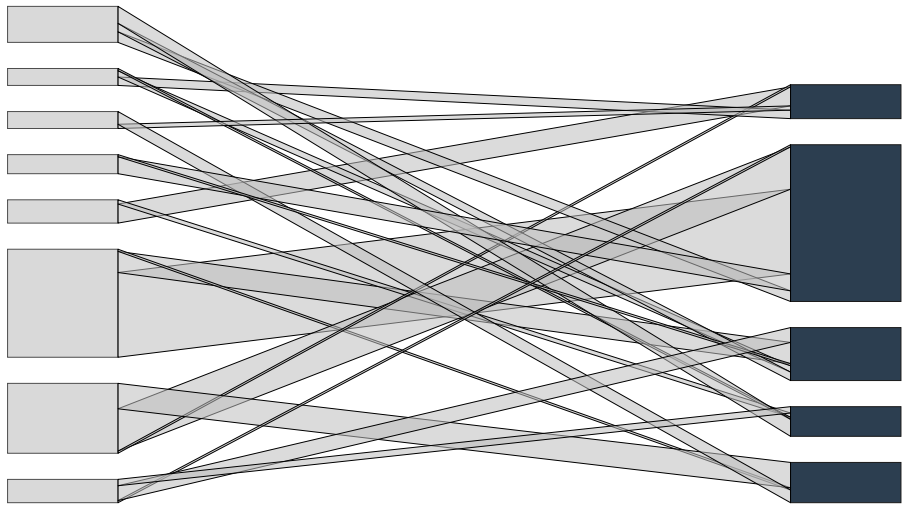
1.5 abundance の基本可視化

この章で使う主な関数:

- geom_bipnet_box(): row / column 側のボックスを描きます。
- geom_bipnet_interaction(): 相互作用ポリゴンを描きます。
- stat_bipnet(): geom_bipnet_*() から内部的に呼ばれ、座標計算を行います。

```
gap_value <- sum(interaction_matrix) / 12

p_abundance <- ggplot(
  interaction_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_box(
    type = "row",
    gap = gap_value,
    fill = "grey85",
    colour = "grey30",
    linewidth = 0.2
```

```
  ) +
  geom_bipnet_box(
    type = "column",
    gap = gap_value,
    fill = "#2C3E50",
    colour = "grey10",
    linewidth = 0.2
  ) +
  geom_bipnet_interaction(
    gap = gap_value,
    alpha = 0.55,
    linewidth = 0.15,
    show.legend = FALSE
  ) +
  theme_void()

p_abundance
```



gap を未指定にすると stat_bipnet() が自動で値を選びます（sum(count) / 10、非正の場合は 1）。
再現性の観点では、gap は明示指定するのが安全です。

補足: geom_bipnet_interaction() では type = "interaction" が既定なので、通常は省略できます（上のコードも省略形です）。

1.6　メタデータで色分けする

この章で使う主な関数:

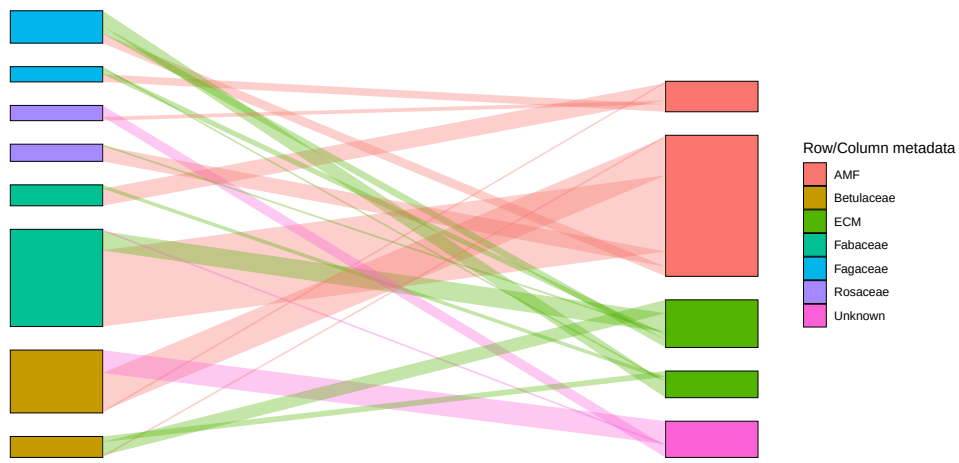
- `geom_bipnet_box()`: row / column のメタデータ色分けに使います。
- `geom_bipnet_interaction()`: 連結部もメタデータで色分けできます。
- `geom_bipnet_point()`: binary 表示時のノード色分けに使えます。
- `after_stat()`: `stat_bipnet()` が後段で付与する列を参照します。

row 側だけでなく column 側でも同様に色分けできます。
さらに同じ考え方で `geom_bipnet_point()` と `geom_bipnet_interaction()` でも色分け可能です。

`after_stat()` を使う理由は、`metadata_row/metadata_column` の列が `stat_bipnet()` の計算後にデータへ結合されるためです。つまり元データ `interaction_df` には存在しない列を 審美属性に使うには `after_stat()` が必要になります。

```
p_metadata <- ggplot(
  interaction_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_box(
    aes(fill = after_stat(family)),
    type = "row",
    row_nm = "host",
    metadata_row = metadata_row,
    gap = gap_value,
    linewidth = 0.2
  ) +
  geom_bipnet_box(
    aes(fill = after_stat(guild)),
    type = "column",
    column_nm = "symbiont",
    metadata_column = metadata_column,
    gap = gap_value,
    linewidth = 0.2
  ) +
  geom_bipnet_interaction(
    aes(fill = after_stat(guild)),
```

```
    column_nm = "symbiont",
    metadata_column = metadata_column,
    gap = gap_value,
    alpha = 0.35,
    linewidth = 0.1,
    colour = NA,
    show.legend = FALSE
  ) +
  theme_void() +
  labs(fill = "Row/Column metadata")

p_metadata
```



geom_bipnet_point() でも同じ発想で色付けできます。

```
binary_matrix <- (interaction_matrix > 0) * 1L

binary_df <- to_longer(binary_matrix, .rowname = "row") |>
  rename(count = interaction) |>
  mutate(
    row = factor(row, levels = rownames(binary_matrix)),
    column = factor(column, levels = colnames(binary_matrix))
  )

ggplot(binary_df, aes(row = row, column = column, count = count)) +
  geom_bipnet_point(
    aes(fill = after_stat(family)),
```

```
    type = "row",
    row_nm = "host",
    metadata_row = metadata_row
  ) +
  geom_bipnet_point(
    aes(fill = after_stat(guild)),
    type = "column",
    column_nm = "symbiont",
    metadata_column = metadata_column
  )
```

1.7　レイアウトパラメータを調整する

この章で使う主な関数:

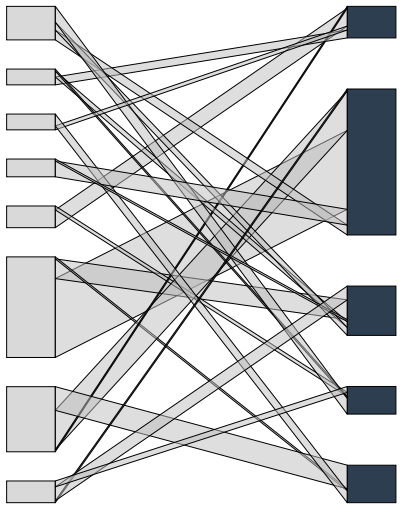
- `geom_bipnet_box()`: `gap`, `box_ratio`, `ratio`, `adjust_box_height` を渡せます。
- `geom_bipnet_interaction()`: 同じレイアウト引数を共有し、各レイヤーの整合を保ちます。
- `stat_bipnet()`: 指定パラメータに基づいて座標を再計算します。

`box_ratio`, `ratio`, `adjust_box_height` は次を調整します。

1. `box_ratio`: `sum(count) / box_ratio` でボックス幅を決める。値を大きくするとボックスは細くなる。
2. `ratio`: 全体の横方向スケールに影響する。値を小さくすると左右の間隔は広がる傾向がある。
3. `adjust_box_height`: `TRUE` で row 側と column 側の総高さを揃えるよう gap を再調整する。

```
p_abundance_tuned <- ggplot(
  interaction_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_box(
    type = "row",
    gap = sum(interaction_matrix) / 10,
    box_ratio = 6,
    ratio = 0.75,
    adjust_box_height = TRUE,
```

```
    fill = "grey85",
    linewidth = 0.2
  ) +
  geom_bipnet_box(
    type = "column",
    gap = sum(interaction_matrix) / 10,
    box_ratio = 6,
    ratio = 0.75,
    adjust_box_height = TRUE,
    fill = "#2C3E50",
    linewidth = 0.2
  ) +
  geom_bipnet_interaction(
    gap = sum(interaction_matrix) / 10,
    box_ratio = 6,
    ratio = 0.75,
    adjust_box_height = TRUE,
    alpha = 0.5,
    linewidth = 0.15,
    show.legend = FALSE
  ) +
  coord_equal() +
  theme_void()

p_abundance_tuned
```



1.8 binary（有無）表示に切り替える

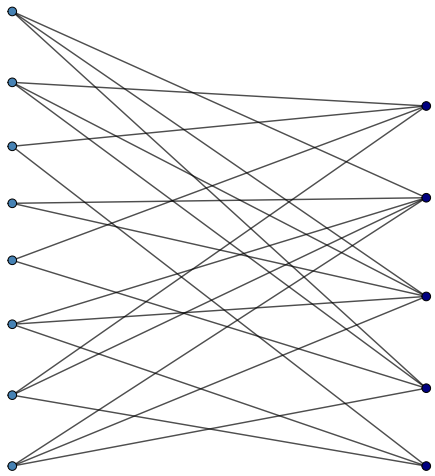
この章で使う主な関数:

- `to_longer()`: binary 行列を long 形式へ変換します。
- `geom_bipnet_interaction(interaction_type = "binary")`: 相互作用を線分として描きます。
- `geom_bipnet_point()`: row / column ノードを点として描きます。

接続の「強さ」ではなく「存在/非存在」を見たい場合は `interaction_type = "binary"` を使います。

```
binary_matrix <- (interaction_matrix > 0) * 1L

binary_df <- to_longer(binary_matrix, .rowname = "row") |>
  rename(count = interaction) |>
  mutate(
    row = factor(row, levels = rownames(binary_matrix)),
    column = factor(column, levels = colnames(binary_matrix))
  )

binary_gap <- sum(binary_matrix) / 10

p_binary <- ggplot(
  binary_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_interaction(
    interaction_type = "binary",
    gap = binary_gap,
    linewidth = 0.45,
    alpha = 0.7,
    show.legend = FALSE
  ) +
  geom_bipnet_point(
    type = "row",
    gap = binary_gap,
    fill = "steelblue",
    size = 2.4
```

```
  ) +
  geom_bipnet_point(
    type = "column",
    gap = binary_gap,
    fill = "navy",
    size = 2.4
  ) +
  coord_equal() +
  theme_void()

p_binary
```



1.9　construct_bn_coordination() でラベル注釈する

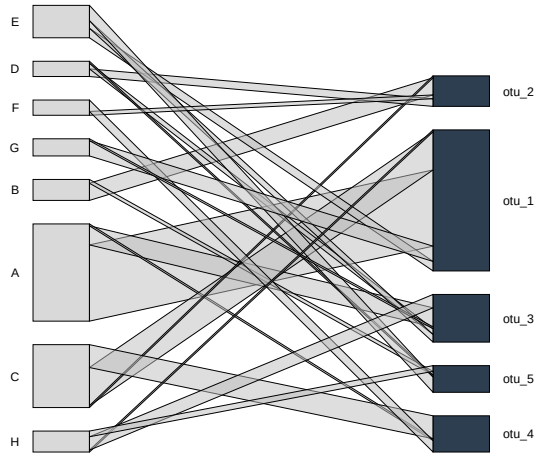
この章で使う主な関数:

- construct_bn_coordination(): row_box / column_box / interaction 座標を明示的に作成します。
- compute_box_label_coords(): ボックス外側ラベルの代表座標を算出します。
- annotate(): 算出座標を使ってテキスト注釈を描きます。

construct_bn_coordination() は通常、geom_bipnet_*() が内部で呼び出している座標計算関数です。
ふだんは直接呼ぶ必要はありませんが、ラベル座標や外部図形（リンク線など）を自作するときには有用です。

```
bn_coords <- construct_bn_coordination(
  .mat = interaction_matrix,
  .row = "host",
  .column = "symbiont",
  .metadata_row = metadata_row,
  .metadata_column = metadata_column,
  .gap = gap_value
)

label_row <- compute_box_label_coords(
  .box = bn_coords$row_box,
  .by = "row",
  .side = "row"
)

label_column <- compute_box_label_coords(
  .box = bn_coords$column_box,
  .by = "column",
  .side = "column"
)

x_span <- diff(range(c(label_row$x, label_column$x)))
label_offset <- x_span * 0.03

ggplot(
  interaction_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_box(type = "row", gap = gap_value, fill = "grey85", linewidth = 0.2) +
  geom_bipnet_box(type = "column", gap = gap_value, fill = "#2C3E50", linewidth = 0.2) +
  geom_bipnet_interaction(
    gap = gap_value,
    alpha = 0.5,
    linewidth = 0.15,
    show.legend = FALSE
  ) +
  annotate(
    geom = "text",
```

```
    x = label_row$x - label_offset,
    y = label_row$y,
    label = label_row$row,
    hjust = 1,
    size = 3
  ) +
  annotate(
    geom = "text",
    x = label_column$x + label_offset,
    y = label_column$y,
    label = label_column$column,
    hjust = 0,
    size = 3
  ) +
  coord_equal(clip = "off") +
  theme_void()
```



1.10　系統樹とネットワークを対応づける

この章で使う主な関数:

- `adjust_tree()`: tree の y スケールをボックスに合わせます。
- `create_link()`: ボックス中心と tip を結ぶリンク座標を作ります。
- `get_yrange()`: 複数パネルの y 範囲を共通化します。
- `geom_tiplab()`: tip ラベル（通常版と `align = TRUE` 版）を描きます。

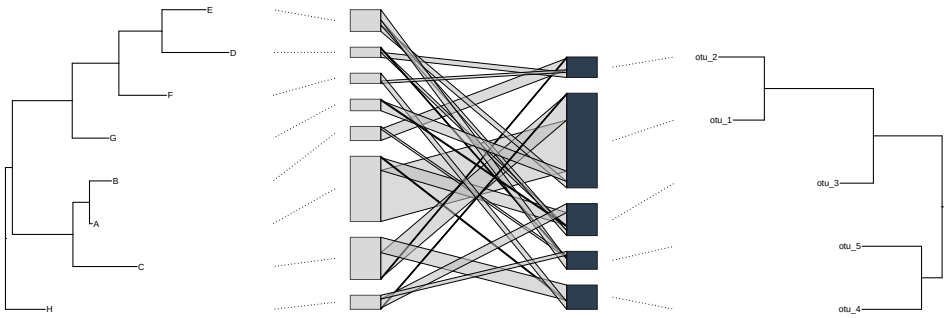
ここでは次の 3 関数を連携させます。

1. `adjust_tree()`: 系統樹の y スケールをボックスに合わせる
2. `create_link()`: ボックス中心と tip を結ぶリンク座標を作る
3. `get_yrange()`: 複数パネルの y 範囲を揃える

```
t_row <- adjust_tree(
  .phylo = host_phylo,
  .box = bn_coords$row_box,
  .tree_position = "left",
  .adjust = 1
) +
  geom_tiplab(size = 3)

t_column <- adjust_tree(
  .phylo = symbiont_phylo,
  .box = bn_coords$column_box,
  .tree_position = "right",
  .adjust = 1
) +
  geom_tiplab(hjust = 1, size = 3)

link_row <- create_link(
  box = bn_coords$row_box,
  ggtree = t_row,
  side = "row",
  x = 1,
  xend = 0
)

link_column <- create_link(
  box = bn_coords$column_box,
  ggtree = t_column,
  side = "column",
  x = 0,
  xend = 1
)

p_link_row <- ggplot(link_row) +
  geom_segment(
```

```
    aes(x = x, y = y1, xend = xend, yend = y2, group = row),
    linetype = "dotted",
    linewidth = 0.4
  ) +
  theme_void()

p_link_column <- ggplot(link_column) +
  geom_segment(
    aes(x = x, y = y1, xend = xend, yend = y2, group = column),
    linetype = "dotted",
    linewidth = 0.4
  ) +
  theme_void()

ylim_common <- range(
  c(get_yrange(t_row), get_yrange(t_column), get_yrange(p_abundance)),
  na.rm = TRUE
)

scale_y_common <- scale_y_continuous(
  limits = ylim_common,
  expand = expansion(mult = 0)
)

t_row <- t_row + scale_y_common
t_column <- t_column + scale_y_common
p_link_row <- p_link_row + scale_y_common
p_link_column <- p_link_column + scale_y_common
p_abundance_aligned <- p_abundance + scale_y_common
```

```
t_row +
  p_link_row +
  p_abundance_aligned +
  p_link_column +
  t_column +
  plot_layout(nrow = 1, widths = c(1, 0.25, 1, 0.25, 1))
```



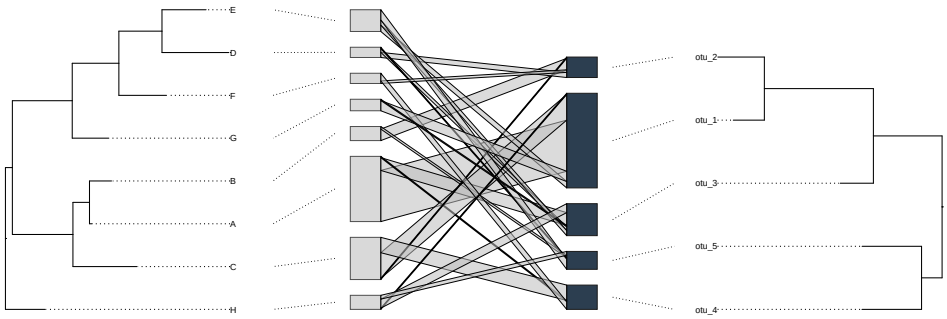
上記のバリエーションとして、tip ラベルを `align = TRUE` で描く版も 用意できます。tip ラベルの x 位置が揃うため、ラベル列の視認性が上がります。

```
t_row_align <- adjust_tree(
  .phylo = host_phylo,
  .box = bn_coords$row_box,
  .tree_position = "left",
  .adjust = 1
) +
  geom_tiplab(
    align = TRUE,
    linetype = "dotted",
    size = 3
  )

t_column_align <- adjust_tree(
  .phylo = symbiont_phylo,
  .box = bn_coords$column_box,
  .tree_position = "right",
  .adjust = 1
) +
  geom_tiplab(
    align = TRUE,
    linetype = "dotted",
    hjust = 1,
    size = 3
  )

link_row_align <- create_link(
  box = bn_coords$row_box,
```

```
  ggtree = t_row_align,
  side = "row",
  x = 1,
  xend = 0
)

link_column_align <- create_link(
  box = bn_coords$column_box,
  ggtree = t_column_align,
  side = "column",
  x = 0,
  xend = 1
)

p_link_row_align <- ggplot(link_row_align) +
  geom_segment(
    aes(x = x, y = y1, xend = xend, yend = y2, group = row),
    linetype = "dotted",
    linewidth = 0.4
  ) +
  theme_void()

p_link_column_align <- ggplot(link_column_align) +
  geom_segment(
    aes(x = x, y = y1, xend = xend, yend = y2, group = column),
    linetype = "dotted",
    linewidth = 0.4
  ) +
  theme_void()

ylim_common_align <- range(
  c(
    get_yrange(t_row_align),
    get_yrange(t_column_align),
    get_yrange(p_abundance)
  ),
  na.rm = TRUE
)
```

```
scale_y_common_align <- scale_y_continuous(
  limits = ylim_common_align,
  expand = expansion(mult = 0)
)

t_row_align <- t_row_align + scale_y_common_align
t_column_align <- t_column_align + scale_y_common_align
p_link_row_align <- p_link_row_align + scale_y_common_align
p_link_column_align <- p_link_column_align + scale_y_common_align
p_abundance_align <- p_abundance + scale_y_common_align
```

```
t_row_align +
  p_link_row_align +
  p_abundance_align +
  p_link_column_align +
  t_column_align +
  patchwork::plot_layout(nrow = 1, widths = c(1, 0.25, 1, 0.25, 1))
```



1.11　応用編: 系統樹スケールを維持した可視化

この章で使う主な関数:

- `geom_bipnet_interaction()`: `tip_positions_row/column` で tree 由来 y 座標に合わせます。
- `geom_bipnet_point()`: binary ノード位置を同じ tip 座標へ固定します。
- `get_yrange()`: tree と network の y 範囲を共通化します。
- `ggtree()`: tip 座標テーブル（`label, y`）抽出に使います。

前節では `adjust_tree()` で系統樹をネットワーク側のスケールに合わせました。
ここでは逆に、系統樹の tip の y 座標を基準にしてネットワーク側を合わせる方法を示し

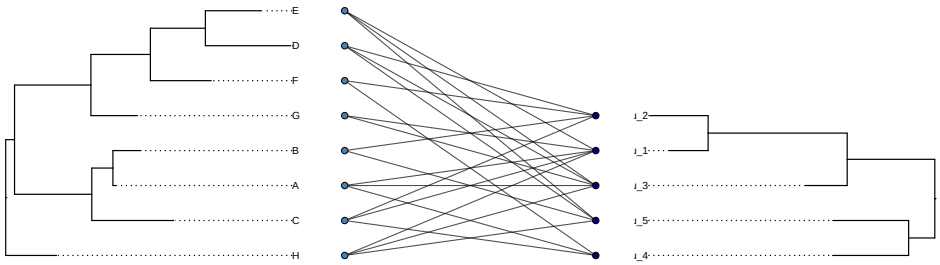
ます。この場合は link が不要です。また、abundance の場合は現実的な調整が難しいため、binary で使う前提です。

```
host_tip_positions <- ggtree(host_phylo)$data |>
  filter(!is.na(isTip) & isTip) |>
  transmute(
    label = as.character(label),
    y = as.numeric(y)
  ) |>
  group_by(label) |>
  summarise(y = mean(y), .groups = "drop")

symbiont_tip_positions <- ggtree(symbiont_phylo)$data |>
  filter(!is.na(isTip) & isTip) |>
  transmute(
    label = as.character(label),
    y = as.numeric(y)
  ) |>
  group_by(label) |>
  summarise(y = mean(y), .groups = "drop")

p_binary_tree_scale <- ggplot(
  binary_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_interaction(
    interaction_type = "binary",
    tip_positions_row = host_tip_positions,
    tip_positions_column = symbiont_tip_positions,
    linewidth = 0.45,
    alpha = 0.7,
    show.legend = FALSE
  ) +
  geom_bipnet_point(
    type = "row",
    tip_positions_row = host_tip_positions,
    tip_positions_column = symbiont_tip_positions,
    fill = "steelblue",
    size = 2.4
```

```
  ) +
  geom_bipnet_point(
    type = "column",
    tip_positions_row = host_tip_positions,
    tip_positions_column = symbiont_tip_positions,
    fill = "navy",
    size = 2.4
  ) +
  theme_void()

t_row_tree_scale <- ggtree(host_phylo) +
  geom_tiplab(
    align = TRUE,
    linetype = "dotted",
    size = 3
  )

t_column_tree_scale <- ggtree(symbiont_phylo) +
  geom_tiplab(
    align = TRUE,
    linetype = "dotted",
    hjust = 1,
    size = 3
  ) +
  scale_x_reverse()

ylim_tree_scale <- range(
  c(
    get_yrange(t_row_tree_scale),
    get_yrange(t_column_tree_scale),
    get_yrange(p_binary_tree_scale)
  ),
  na.rm = TRUE
)

scale_y_tree_scale <- scale_y_continuous(
  limits = ylim_tree_scale,
  expand = expansion(mult = 0)
```

```
)

t_row_tree_scale <- t_row_tree_scale + scale_y_tree_scale
t_column_tree_scale <- t_column_tree_scale + scale_y_tree_scale
p_binary_tree_scale <- p_binary_tree_scale + scale_y_tree_scale
```

```
t_row_tree_scale +
  p_binary_tree_scale +
  t_column_tree_scale +
  plot_layout(nrow = 1, widths = c(1, 1, 1))
```



1.12　よくあるつまずき

この章で参照する主な関数:

- `get_tip_order()`: tree と行列の順序不一致を確認します。
- `after_stat()`: metadata 列参照に必要な点を再確認します。
- `geom_bipnet_*()`: `gap`, `row_nm`, `column_nm` の指定ミスを切り分けます。

1. 系統樹の tip ラベルと行列の行名・列名が一致していない
   `get_tip_order()` の返り値と `rownames()` / `colnames()` を照合する。
2. `factor(levels = ...)` と行列ソートを行っていない
   順序がズレて tree と network の対応が崩れる。
3. レイヤーごとに `gap` が違う
   ボックスと相互作用の位置がズレるため、同じ値を使う。
4. metadata のキー名が `row_nm` / `column_nm` と一致していない
   `after_stat()` で参照する列が生成されない。

1.13　まとめ

この章で再確認する主要関数:

- `to_longer()`
- `geom_bipnet_box()`
- `geom_bipnet_interaction()`
- `geom_bipnet_point()`
- `adjust_tree()` / `create_link()`

`ggbipartite` は、次の順番で使うと安定します。

1. 先に系統樹を描いて左右の意味を固定する
2. `factor` と行列ソートで順序を揃える
3. `row` / `column` / `count` を明示して基本図を作る
4. 必要に応じて metadata・binary・系統樹連携へ拡張する

この vignette をテンプレートにすれば、実データにもそのまま移植できます。

第 2 章

ggbipartite で作る Publishable な系統樹 × 二者ネットワーク図

この vignette では、`ggbipartite` と `ggtree` を組み合わせて、投稿図として使いやすい品質の図を作る発展例を示します。

扱う内容:

- `align = TRUE` の tip 表示
- `geom_treescale()` によるスケール表示
- `geom_nodelab()` による bootstrap 値表示
- `marquee` による tip / node ラベルの文字スタイル統一
- abundance 版と binary 版（系統樹スケール維持）の両方
- `phylosignal_sub_network()` 結果を `geom_nodepoint()` で重ねる例

2.1 事前準備

```
library(tidyverse)
library(cowplot)
library(ape)
library(ggtree)
library(marquee)
library(RPANDA)
library(ggbipartite)
```

2.2　データと系統樹を作る

この章で使う主な関数:

- `rtree()`: 例示用の系統樹を作成します。
- `get_tip_order()`: 系統樹と行列の並び順を揃えます。
- `to_longer()`: 行列を `row-column-count` 形式へ変換します。

```
set.seed(20260304)

host_phylo <- rtree(10)
host_phylo$tip.label <- paste0("Host_", sprintf("%02d", 1:10))
host_phylo$node.label <- sample(70:100, host_phylo$Nnode, replace = TRUE)

symbiont_phylo <- rtree(6)
symbiont_phylo$tip.label <- paste0("OTU_", sprintf("%02d", 1:6))
symbiont_phylo$node.label <- sample(70:100, symbiont_phylo$Nnode, replace = TRU

host_order <- get_tip_order(host_phylo) |> rev()
symbiont_order <- get_tip_order(symbiont_phylo) |> rev()

interaction_matrix_raw <- matrix(
  c(
    120, 0, 40, 0, 15, 0,
    40, 2, 0, 0, 6, 1,
    0, 12, 0, 4, 0, 0,
    8, 0, 0, 22, 0, 5,
    0, 0, 5, 8, 18, 0,
    65, 0, 20, 0, 3, 0,
    0, 7, 2, 0, 0, 13,
    4, 0, 0, 30, 0, 0,
    0, 0, 8, 3, 25, 2,
    20, 1, 0, 0, 10, 0
  ),
  nrow = 10,
  byrow = TRUE,
  dimnames = list(
    paste0("Host_", sprintf("%02d", 1:10)),
```

```
    paste0("OTU_", sprintf("%02d", 1:6))
  )
)

interaction_matrix <- interaction_matrix_raw[
  host_order,
  symbiont_order,
  drop = FALSE
]

metadata_row <- tibble(
  host = rownames(interaction_matrix),
  host_lineage = c(rep("Lineage_A", 6), rep("Lineage_B", 4))
)

metadata_row
```

```
# A tibble: 10 x 2
   host    host_lineage
   <chr>   <chr>
 1 Host_10 Lineage_A
 2 Host_09 Lineage_A
 3 Host_08 Lineage_A
 4 Host_07 Lineage_A
 5 Host_06 Lineage_A
 6 Host_05 Lineage_A
 7 Host_04 Lineage_B
 8 Host_03 Lineage_B
 9 Host_02 Lineage_B
10 Host_01 Lineage_B
```

```
metadata_column <- tibble(
  symbiont = colnames(interaction_matrix),
  guild = c("AMF", "AMF", "ECM", "ECM", "Saprotroph", "Unknown")
)

metadata_column
```

```
# A tibble: 6 x 2
```

```
  symbiont guild
  <chr>    <chr>
1 OTU_04   AMF
2 OTU_03   AMF
3 OTU_05   ECM
4 OTU_02   ECM
5 OTU_01   Saprotroph
6 OTU_06   Unknown
```

```
interaction_df <- to_longer(interaction_matrix, .rowname = "row") |>
  rename(count = interaction) |>
  mutate(
    row = factor(row, levels = rownames(interaction_matrix)),
    column = factor(column, levels = colnames(interaction_matrix))
  )

binary_matrix <- (interaction_matrix > 0) * 1L

binary_df <- to_longer(binary_matrix, .rowname = "row") |>
  rename(count = interaction) |>
  mutate(
    row = factor(row, levels = rownames(binary_matrix)),
    column = factor(column, levels = colnames(binary_matrix))
  )
```

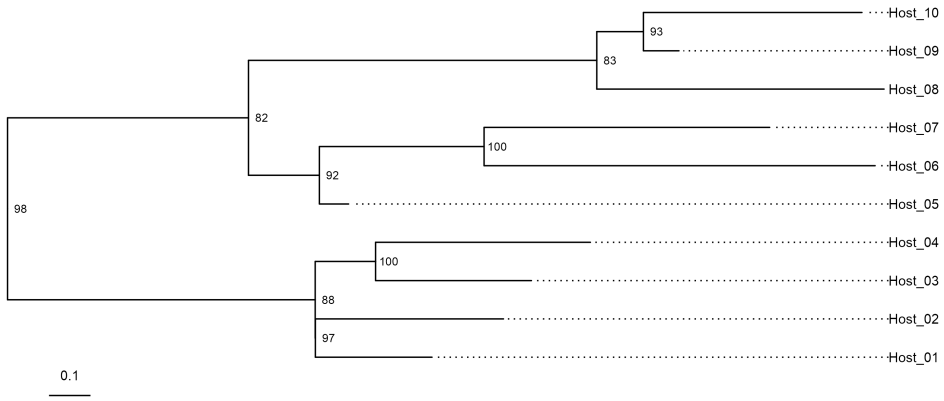
2.3　系統樹を注釈する

この章で使う主な関数:

- `geom_tiplab(align = TRUE)`: tip ラベルを整列表示します。
- `geom_treescale()`: 枝長スケールを図に追加します。
- `geom_nodelab()`: 内部ノード bootstrap 値を表示します。

```
label_family <- "Arial"
tip_text_size <- 3.5
node_text_size <- 3

host_tree_base <- ggtree(host_phylo)
host_tree_span <- diff(range(host_tree_base$data$x, na.rm = TRUE))
```

```
host_tree_annotated <- host_tree_base +
  geom_tiplab(
    align = TRUE,
    linetype = "dotted",
    family = label_family,
    size = tip_text_size
  ) +
  geom_nodelab(
    aes(label = label),
    node = "internal",
    family = label_family,
    size = node_text_size,
    colour = "black",
    nudge_x = host_tree_span * 0.01,
    hjust = 0.5
  ) +
  geom_treescale(
    x = 0.1,
    y = 0,
    width = 0.1,
    fontsize = tip_text_size,
    family = label_family
  ) +
  coord_cartesian(clip = "off")

host_tree_annotated
```



2.4 marquee で tip/node ラベルの見た目を揃える

この章で使う主な関数:

- `adjust_tree()`: tree を bipartite ボックスへ整列します。
- `.tiplab_segment_data()`: align 用の補助線座標を作ります。
- `.tiplab_marquee_data()`: `geom_marquee()` 用 tip 座標を作ります。
- `geom_marquee()`: tip ラベルを同一フォントで安定描画します。

以降のコードでは、関数を定義せずに各チャンク内で直接 `adjust_tree()` / `geom_marquee()` / `geom_nodelab()` / `geom_treescale()` を組み合わせます。

2.5 abundance 版の図を作る

この章で使う主な関数:

- `construct_bn_coordination()`: row / column / interaction 座標を作成します。
- `geom_bipnet_box()`: row / column ボックスを描きます。
- `geom_bipnet_interaction()`: abundance ポリゴンを描きます。
- `create_link()`: tree と box を結ぶリンク座標を作ります。
- `get_yrange()`: パネル間の y 軸を統一します。

```
gap_abundance <- sum(interaction_matrix) / 12

bn_coords_abundance <- construct_bn_coordination(
  .mat = interaction_matrix,
  .row = "host",
  .column = "symbiont",
  .metadata_row = metadata_row,
  .metadata_column = metadata_column,
  .gap = gap_abundance
)

t_host_abundance_base <- adjust_tree(
  .phylo = host_phylo,
  .box = bn_coords_abundance$row_box,
  .adjust = 1,
  .tree_position = "left"
)
```

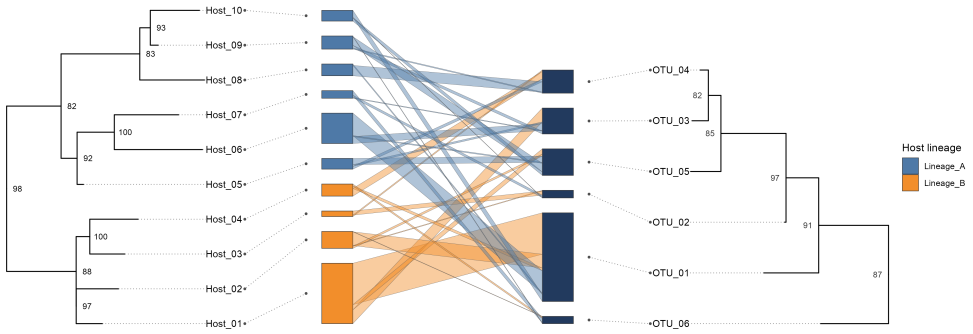
```
x_span_host_abundance <- diff(range(t_host_abundance_base$data$x, na.rm = TRUE))

t_host_abundance <- t_host_abundance_base +
  geom_segment(
    data = function(df) .tiplab_segment_data(df, offset = 0),
    aes(x = x, xend = xend, y = y, yend = yend),
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) .tiplab_marquee_data(df, offset = 0),
    aes(x = x_lab, y = y, label = label),
    inherit.aes = FALSE,
    hjust = 0,
    family = label_family,
    size = tip_text_size
  ) +
  geom_nodelab(
    aes(label = label),
    node = "internal",
    nudge_x = x_span_host_abundance * 0.05,
    hjust = 0.5,
    family = label_family,
    size = node_text_size
  ) +
  coord_cartesian(clip = "off")

t_symbiont_abundance_base <- adjust_tree(
  .phylo = symbiont_phylo,
  .box = bn_coords_abundance$column_box,
  .adjust = 1,
  .tree_position = "right"
)

x_span_symbiont_abundance <- diff(range(t_symbiont_abundance_base$data$x, na.rm = TRUE))
```

```
t_symbiont_abundance <- t_symbiont_abundance_base +
  geom_segment(
    data = function(df) .tiplab_segment_data(df, offset = 0),
    aes(x = x, xend = xend, y = y, yend = yend),
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) .tiplab_marquee_data(df, offset = 0),
    aes(x = x_lab, y = y, label = label),
    inherit.aes = FALSE,
    hjust = 1,
    family = label_family,
    size = tip_text_size
  ) +
  geom_nodelab(
    aes(label = label),
    node = "internal",
    nudge_x = -x_span_symbiont_abundance * 0.05,
    hjust = 0.5,
    family = label_family,
    size = node_text_size,
    colour = "grey20"
  ) +
  coord_cartesian(clip = "off")

link_host <- create_link(
  box = bn_coords_abundance$row_box,
  ggtree = t_host_abundance,
  side = "row",
  x = 1,
  xend = 0
)

link_symbiont <- create_link(
  box = bn_coords_abundance$column_box,
```

```
  ggtree = t_symbiont_abundance,
  side = "column",
  x = 0,
  xend = 1
)

p_link_host <- ggplot(link_host) +
  geom_segment(
    aes(x = x, y = y1, xend = xend, yend = y2, group = row),
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45"
  ) +
  geom_point(aes(x = x, y = y1), size = 0.7, colour = "grey30") +
  geom_point(aes(x = xend, y = y2), size = 0.7, colour = "grey30") +
  theme_void()

p_link_symbiont <- ggplot(link_symbiont) +
  geom_segment(
    aes(x = x, y = y1, xend = xend, yend = y2, group = column),
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45"
  ) +
  geom_point(aes(x = x, y = y1), size = 0.7, colour = "grey30") +
  geom_point(aes(x = xend, y = y2), size = 0.7, colour = "grey30") +
  theme_void()

p_abundance_publishable <- ggplot(
  interaction_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_box(
    aes(fill = after_stat(host_lineage)),
    type = "row",
    row_nm = "host",
    metadata_row = metadata_row,
    gap = gap_abundance,
```

```
    linewidth = 0.25,
    colour = "grey20"
  ) +
  geom_bipnet_box(
    type = "column",
    gap = gap_abundance,
    fill = "#223A5E",
    linewidth = 0.25,
    colour = "grey20"
  ) +
  geom_bipnet_interaction(
    aes(fill = after_stat(host_lineage)),
    row_nm = "host",
    metadata_row = metadata_row,
    gap = gap_abundance,
    alpha = 0.45,
    linewidth = 0.1,
    colour = "grey35",
    show.legend = FALSE
  ) +
  scale_fill_manual(
    name = "Host lineage",
    values = c("Lineage_A" = "#4E79A7", "Lineage_B" = "#F28E2B")
  ) +
  theme_void(base_family = label_family)

legend <- get_legend(p_abundance_publishable)
p_abundance_publishable <- p_abundance_publishable + theme(legend.position = "n

ylim_abundance <- range(
  c(
    get_yrange(t_host_abundance),
    get_yrange(t_symbiont_abundance),
    get_yrange(p_abundance_publishable)
  ),
  na.rm = TRUE
)
```

```
scale_y_abundance <- scale_y_continuous(
  limits = ylim_abundance,
  expand = expansion(mult = 0)
)

t_host_abundance <- t_host_abundance + scale_y_abundance
t_symbiont_abundance <- t_symbiont_abundance + scale_y_abundance
p_link_host <- p_link_host + scale_y_abundance
p_link_symbiont <- p_link_symbiont + scale_y_abundance
p_abundance_publishable <- p_abundance_publishable + scale_y_abundance

t_host_abundance +
  p_link_host +
  p_abundance_publishable +
  p_link_symbiont +
  t_symbiont_abundance +
  legend +
  patchwork::plot_layout(nrow = 1, widths = c(1, 0.28, 1.15, 0.28, 1, 0.2))
```



2.6 binary 版の図を作る

この章で使う主な関数:

- `geom_bipnet_interaction(interaction_type = "binary")`: 二値相互作用を線分として描画します。
- `geom_bipnet_point()`: row / column ノードを点で描画します。
- `tip_positions_row` / `tip_positions_column`: 系統樹由来の y 座標を直接ネットワークへ適用します。
- `metadata_row` + `after_stat(host_lineage)`: binary でも host lineage による

色分けが可能です。

- `ggtree()`: 枝長スケールを保持した tree パネルを構築します。

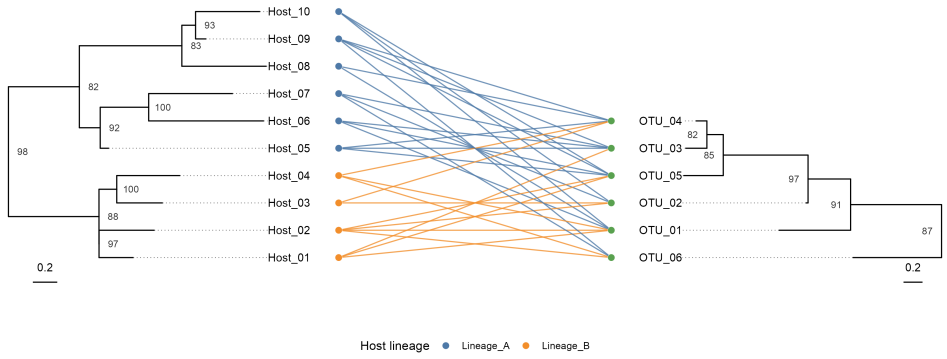
```
host_tip_positions <- ggtree(host_phylo)$data |>
  filter(!is.na(.data$isTip) & .data$isTip) |>
  transmute(
    label = as.character(.data$label),
    y = as.numeric(.data$y)
  ) |>
  group_by(.data$label) |>
  summarise(y = mean(.data$y), .groups = "drop")

symbiont_tip_positions <- ggtree(symbiont_phylo)$data |>
  filter(!is.na(.data$isTip) & .data$isTip) |>
  transmute(
    label = as.character(.data$label),
    y = as.numeric(.data$y)
  ) |>
  group_by(.data$label) |>
  summarise(y = mean(.data$y), .groups = "drop")

p_binary_publishable <- ggplot(
  binary_df,
  aes(row = row, column = column, count = count)
) +
  geom_bipnet_interaction(
    aes(colour = after_stat(host_lineage)),
    interaction_type = "binary",
    row_nm = "host",
    metadata_row = metadata_row,
    tip_positions_row = host_tip_positions,
    tip_positions_column = symbiont_tip_positions,
    linewidth = 0.5,
    alpha = 0.8,
    show.legend = FALSE
  ) +
  geom_bipnet_point(
    aes(fill = after_stat(host_lineage)),
    type = "row",
```

```
    row_nm = "host",
    metadata_row = metadata_row,
    tip_positions_row = host_tip_positions,
    tip_positions_column = symbiont_tip_positions,
    colour = "white",
    stroke = 0.2,
    size = 2.8
  ) +
  geom_bipnet_point(
    type = "column",
    tip_positions_row = host_tip_positions,
    tip_positions_column = symbiont_tip_positions,
    fill = "#59A14F",
    colour = "white",
    stroke = 0.2,
    size = 2.8
  ) +
  scale_colour_manual(
    values = c("Lineage_A" = "#4E79A7", "Lineage_B" = "#F28E2B"),
    guide = "none"
  ) +
  scale_fill_manual(
    values = c("Lineage_A" = "#4E79A7", "Lineage_B" = "#F28E2B"),
    name = "Host lineage"
  ) +
  theme_void(base_family = label_family) +
  theme(
    plot.margin = margin(5.5, 15, 5.5, 15),
    legend.position = "bottom"
  )

t_host_binary_base <- ggtree(host_phylo)
x_span_host_binary <- diff(range(t_host_binary_base$data$x, na.rm = TRUE))

t_host_binary <- t_host_binary_base +
  geom_segment(
    data = function(df) .tiplab_segment_data(df, offset = 0),
    aes(x = x, xend = xend, y = y, yend = yend),
```

```
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) .tiplab_marquee_data(df, offset = 0),
    aes(x = x_lab, y = y, label = label),
    inherit.aes = FALSE,
    hjust = 0,
    family = label_family,
    size = tip_text_size
  ) +
  geom_nodelab(
    aes(label = label),
    node = "internal",
    nudge_x = x_span_host_binary * 0.05,
    hjust = 0.5,
    family = label_family,
    size = node_text_size,
    colour = "grey20"
  ) +
  geom_treescale(
    x = 0.2,
    y = 0.1,
    width = 0.2,
    fontsize = tip_text_size,
    family = label_family
  ) +
  coord_cartesian(clip = "off")

t_symbiont_binary_base <- ggtree(symbiont_phylo)
x_span_symbiont_binary <- diff(range(t_symbiont_binary_base$data$x, na.rm = TRU

t_symbiont_binary <- t_symbiont_binary_base +
  geom_segment(
    data = function(df) .tiplab_segment_data(df, offset = 0),
    aes(x = x, xend = xend, y = y, yend = yend),
```

```
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) .tiplab_marquee_data(df, offset = 0),
    aes(x = x_lab, y = y, label = label),
    inherit.aes = FALSE,
    hjust = 1,
    family = label_family,
    size = tip_text_size
  ) +
  geom_nodelab(
    aes(label = label),
    node = "internal",
    nudge_x = -x_span_symbiont_binary * 0.05,
    hjust = 0.5,
    family = label_family,
    size = node_text_size,
    colour = "grey20"
  ) +
  geom_treescale(
    x = -0.4,
    y = 0.1,
    width = 0.2,
    fontsize = tip_text_size,
    family = label_family
  ) +
  coord_cartesian(clip = "off") +
  scale_x_reverse()

ylim_binary <- range(
  c(
    get_yrange(t_host_binary),
    get_yrange(t_symbiont_binary),
    get_yrange(p_binary_publishable)
  ),
```

```
  na.rm = TRUE
)

scale_y_binary <- scale_y_continuous(
  limits = ylim_binary,
  expand = expansion(mult = 0)
)

t_host_binary <- t_host_binary + scale_y_binary
t_symbiont_binary <- t_symbiont_binary + scale_y_binary
p_binary_publishable <- p_binary_publishable + scale_y_binary

t_host_binary +
  p_binary_publishable +
  t_symbiont_binary +
  patchwork::plot_layout(nrow = 1, widths = c(1, 1.2, 1))
```



2.7　binary + RPANDA シグナルを重ねる (geom_nodepoint())

この章で使う主な関数:

- `phylosignal_sub_network()`: clade 単位の相関検定を実行する関数（本章では実行せず参照のみ）。
- `tribble()`: デモ用の clade シグナル表をコード内で直接作成します。
- `geom_nodepoint()`: 検定結果（`mantel_cor`）を内部ノードへ重ね描きします。

```r
# Example call (not executed):
# phylosignal_sub_network(
#   network = binary_matrix,
#   tree_A = host_phylo,
#   tree_B = symbiont_phylo
# )

cophylo_by_clade_plants <- tribble(
  ~data_source, ~node, ~nb_A, ~nb_B, ~mantel_cor, ~pvalue_upper, ~pvalue_lower, ~pvalue_upper_corrected,
  "ecm_erm", 11, 5, 4, 0.62, 0.0010, 0.2400, 0.0040, 0.4400,
  "ecm_erm", 12, 4, 3, -0.38, 0.0120, 0.4100, 0.0300, 0.6500,
  "ecm_erm", 14, 6, 5, 0.81, 0.0005, 0.1900, 0.0020, 0.3300,
  "ecm_erm", 16, 3, 2, 0.25, 0.0800, 0.5000, 0.1200, 0.7700,
  "rus", 15, 4, 4, -0.55, 0.0300, 0.0100, 0.0700, 0.0300,
  "ecm_erm", 18, 2, 2, -0.67, 0.0020, 0.6600, 0.0060, 0.8900
)

cophylo_by_clade_plants_sigif_upper <- cophylo_by_clade_plants |>
  filter(
    data_source == "ecm_erm",
    pvalue_upper_corrected < 0.05
  )

cophylo_by_clade_plants_sigif_upper
```

```
# A tibble: 4 x 9
  data_source  node  nb_A  nb_B mantel_cor pvalue_upper pvalue_lower
  <chr>       <dbl> <dbl> <dbl>      <dbl>        <dbl>        <dbl>
1 ecm_erm        11     5     4       0.62       0.001          0.24
2 ecm_erm        12     4     3      -0.38       0.012          0.41
3 ecm_erm        14     6     5       0.81       0.0005         0.19
4 ecm_erm        18     2     2      -0.67       0.002          0.66
# i 2 more variables: pvalue_upper_corrected <dbl>,
#   pvalue_lower_corrected <dbl>
```

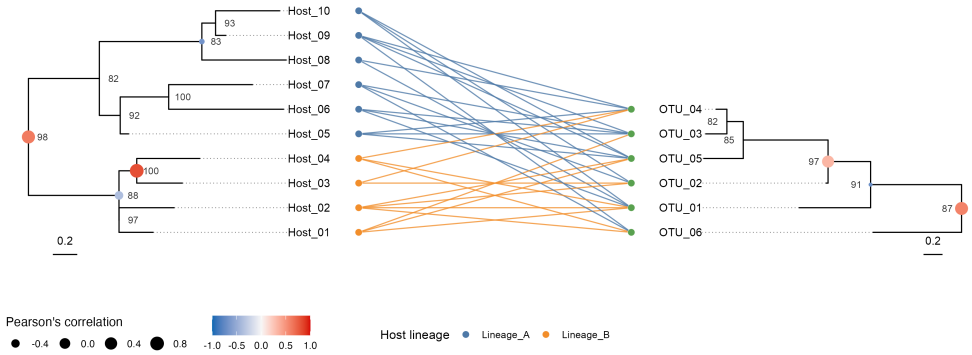
```r
cophylo_by_clade_fungi <- tribble(
  ~data_source, ~node, ~nb_A, ~nb_B, ~mantel_cor, ~pvalue_upper, ~pvalue_lower, ~pvalue_upper_corrected,
  "ecm_erm", 7, 4, 4, 0.58, 0.0040, 0.2000, 0.0100, 0.3100,
  "ecm_erm", 8, 3, 3, -0.71, 0.0008, 0.5000, 0.0030, 0.8000,
```

```
  "ecm_erm", 9, 2, 2, 0.33, 0.0300, 0.4100, 0.0490, 0.7500,
  "ecm_erm", 10, 2, 2, 0.12, 0.2000, 0.6000, 0.2900, 0.8800,
  "rus", 11, 2, 3, -0.47, 0.0400, 0.0300, 0.0900, 0.0500
)

cophylo_by_clade_fungi_sigif_upper <- cophylo_by_clade_fungi |>
  filter(
    data_source == "ecm_erm",
    pvalue_upper_corrected < 0.05
  )

cophylo_by_clade_fungi_sigif_upper
```

```
# A tibble: 3 x 9
  data_source  node  nb_A  nb_B mantel_cor pvalue_upper pvalue_lower
  <chr>       <dbl> <dbl> <dbl>      <dbl>        <dbl>        <dbl>
1 ecm_erm         7     4     4       0.58       0.004          0.2
2 ecm_erm         8     3     3      -0.71       0.0008         0.5
3 ecm_erm         9     2     2       0.33       0.03           0.41
# i 2 more variables: pvalue_upper_corrected <dbl>,
#   pvalue_lower_corrected <dbl>
```

```
cor_limits <- range(
  c(
    cophylo_by_clade_plants_sigif_upper$mantel_cor,
    cophylo_by_clade_fungi_sigif_upper$mantel_cor
  ),
  na.rm = TRUE
)
if (diff(cor_limits) == 0) {
  cor_limits <- cor_limits + c(-1e-6, 1e-6)
}

common_scale <- list(
  scale_color_gradient2(
    name = NULL,
    low = "#0568b3",
    mid = "#F7F7F7",
    high = "#d60205",
```

```
    midpoint = 0,
    limits = c(-1, 1),
    breaks = c(-1, -0.5, 0, 0.5, 1)
  ),
  scale_size(
    name = "Pearson's correlation",
    range = c(1, 5),
    limits = cor_limits
  )
)

t_host_binary_signal <- t_host_binary %<+%
  cophylo_by_clade_plants_sigif_upper +
  geom_nodepoint(aes(size = mantel_cor, colour = mantel_cor)) +
  common_scale +
  theme(
    legend.position = "bottom",
    legend.title.position = "top"
  )

t_symbiont_binary_signal <- t_symbiont_binary %<+%
  cophylo_by_clade_fungi_sigif_upper +
  geom_nodepoint(aes(size = mantel_cor, colour = mantel_cor)) +
  common_scale +
  theme(legend.position = "none")

t_host_binary_signal +
  p_binary_publishable +
  t_symbiont_binary_signal +
  patchwork::plot_layout(nrow = 1, widths = c(1, 1.2, 1))
```



2.8　出力時の実務メモ

この章で使う主な関数:

- `ggsave()`: 論文図として保存します。
- `device = cairo_pdf`: 文字埋め込みや線描画を安定化できます。

```
ggsave(
  filename = "plots/network.pdf",
  plot = p_otu_composition,
  device = grDevices::cairo_pdf,
  width = 9.6,
  height = 7.2,
  units = "in",
  limitsize = FALSE,
  bg = "transparent"
)
```

第 3 章

学名付きの系統樹ラベルを整える

この vignette では、学名付きラベルを読みやすく整形しつつ、`ggtree` で tip label を綺麗に配置する最小例を示します。

学名部分はすべてダミーデータです。

3.1 事前準備

この章で使う主な関数:

- `style_tree_label()`: 学名ラベルを Markdown 形式で整形します。
- `format_node_support()`: node support 値を閾値で整形します。
- `geom_marquee()`: Markdown ラベルを安定して描画します。
- `.tiplab_*()`: tip label の x 座標と補助線を計算します。

```
library(tidyverse)
library(ape)
library(ggtree)
library(marquee)
library(ggbipartite)
```

3.2 ダミー系統樹とラベルを用意する

```
set.seed(20260305)

tree_raw <- rtree(10)
tree_raw$tip.label <- c(
  "Cortinarius-violaceus_AB123456.1",
```

```
  "Laccaria-bicolor_MN234567.1",
  "Russula-cyanoxantha_LC345678.1",
  "mycobiont_of_Fagus-crenata_ORM_OTU1_Psathyrella",
  "mycobiont_of_Quercus-crispula_ORM_OTU2_Tomentella",
  "mycobiont_of_Betula-ermanii_ORM_OTU3_Thelephoraceae",
  "Amanita-muscaria_OP456789.1",
  "Clavulina-cinerea_PP567890.1",
  "Inocybe-geophylla_QQ678901.1",
  "Hebeloma-crustuliniforme_RR789012.1"
)

support_pool <- c(
  "98/100", "95/99", "92/97", "88/96", "84/95",
  "79/90", "70/86", "66/82", "61/80"
)
tree_raw$node.label <- support_pool[seq_len(tree_raw$Nnode)]

tree_rooted <- root(
  phy = tree_raw,
  outgroup = tree_raw$tip.label[1],
  resolve.root = TRUE
)
```

3.3　学名とサポート値を整形する

```
tree_pretty <- tree_rooted

tree_pretty$tip.label <- tree_rooted$tip.label |>
  str_replace_all("_ORM_", "_(ORM)_") |>
  str_remove("\\.1$") |>
  style_tree_label(last_taxon_rank = "genus")

tree_pretty$node.label <- format_node_support(
  x = tree_rooted$node.label,
  sh_alrt_cutoff = 80,
  boot_cutoff = 95
)
```

```
label_preview <- tibble(
  raw_label = tree_rooted$tip.label,
  styled_label = tree_pretty$tip.label
)

label_preview
```

```
# A tibble: 10 x 2
   raw_label                                          styled_label
   <chr>                                              <chr>
 1 Cortinarius-violaceus_AB123456.1                   *Cortinarius* *violaceus~
 2 Laccaria-bicolor_MN234567.1                        *Laccaria* *bicolor* MN2~
 3 Russula-cyanoxantha_LC345678.1                     *Russula* *cyanoxantha* ~
 4 mycobiont_of_Fagus-crenata_ORM_OTU1_Psathyrella    mycobiont of *Fagus* *cr~
 5 mycobiont_of_Quercus-crispula_ORM_OTU2_Tomentella  mycobiont of *Quercus* *~
 6 mycobiont_of_Betula-ermanii_ORM_OTU3_Thelephoraceae mycobiont of *Betula* *e~
 7 Amanita-muscaria_OP456789.1                        *Amanita* *muscaria* OP4~
 8 Clavulina-cinerea_PP567890.1                       *Clavulina* *cinerea* PP~
 9 Inocybe-geophylla_QQ678901.1                       *Inocybe* *geophylla* QQ~
10 Hebeloma-crustuliniforme_RR789012.1                *Hebeloma* *crustulinifo~
```

実務では、系統推定結果を phylo として読み込み、必要なら root を設定したうえで tibble に変換し、label 列を mutate() + case_when() で整形してから phylo に戻す流れが扱いやすいです。
mycobiont・サポート値・その他の学名を 1 箇所で分岐できるため、可視化前の前処理を集約できます。

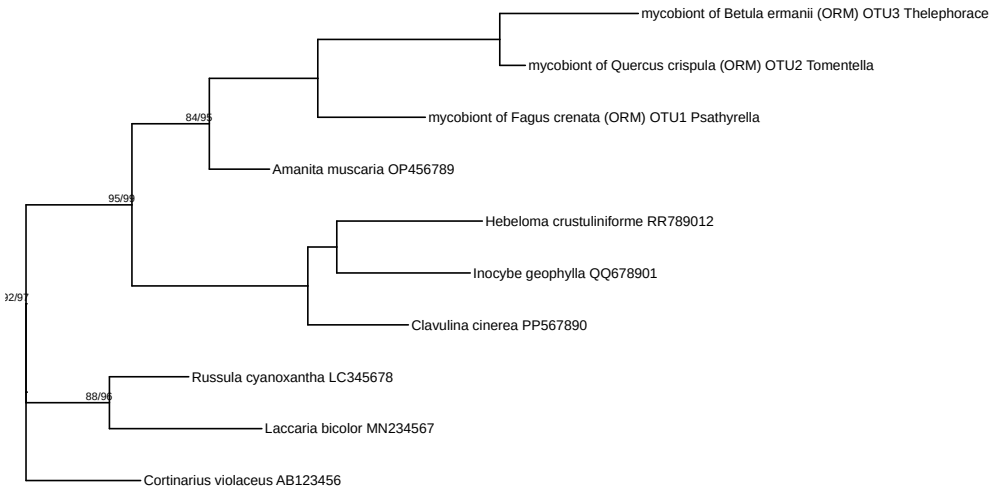
```
tree_pretty_from_file <- read.tree("path/to/your_tree.treefile") |>
  phytools::reroot(node.number = 31) |> # Set root at a target clade.
  as_tibble() |>
  mutate(
    label = str_replace_all(
      .data$label,
      "_ORM_",
      "(ORM)"
    ),
    label = str_remove(.data$label, "\\.1$"),
    label = case_when(
      str_detect(.data$label, "mycobiont") ~
```

```
        style_tree_label(.data$label),
      str_detect(
        .data$label,
        "^\\d+(?:\\.\\d+)?/\\d+(?:\\.\\d+)?$"
      ) ~ .data$label,
      .default = style_sciname(.data$label)
    )
  ) |>
  as.phylo()
```

この方法だと、ggtree() 側では整形済みラベルをそのまま使えるため、プロット層で文字列処理を書く必要が減り、図コードの可読性が上がります。

3.4　align = FALSE: tip ごとにラベルを寄せる

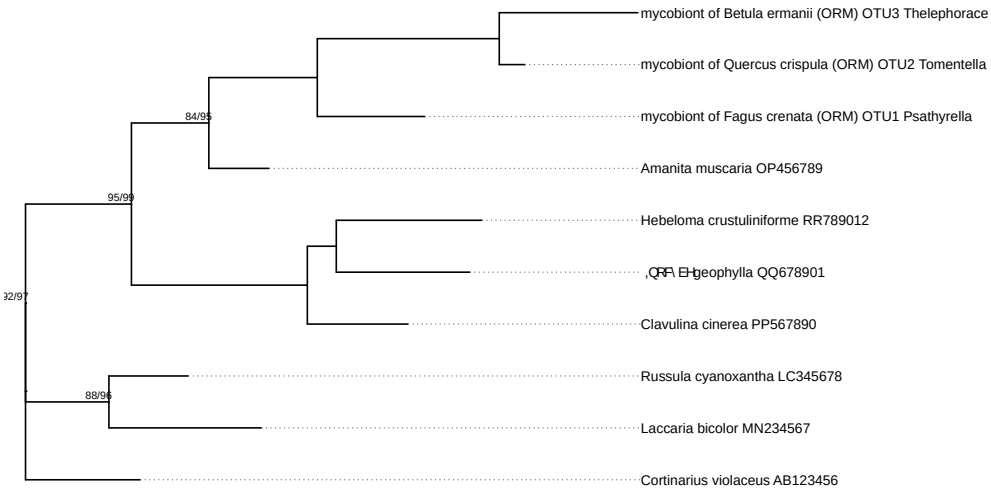
```
base_tree <- ggtree(tree_pretty) +
  scale_x_continuous(expand = expansion(mult = c(0.02, 0.55)))

p_unaligned <- base_tree +
  geom_marquee(
    data = function(df) {
      .tiplab_marquee_data_unaligned(df = df, offset = 0)
    },
    aes(x = x_lab, y = y, label = label),
    hjust = 0,
    size = 3.2,
    family = "Arial",
    inherit.aes = FALSE
  ) +
  geom_nodelab(
    aes(label = label),
    hjust = 1,
    vjust = -0.4,
    size = 2.6
  ) +
  coord_cartesian(clip = "off")

p_unaligned
```



3.5 align = TRUE: 補助線を付けて整列する

```
p_aligned <- base_tree +
  geom_segment(
    data = function(df) {
      .tiplab_segment_data(df = df, offset = 0)
    },
    aes(x = x, xend = xend, y = y, yend = yend),
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) {
      .tiplab_marquee_data(df = df, offset = 0)
    },
    aes(x = x_lab, y = y, label = label),
    hjust = 0,
    size = 3.2,
    family = "Arial",
    inherit.aes = FALSE
  ) +
  geom_nodelab(
```

```
    aes(label = label),
    hjust = 1,
    vjust = -0.4,
    size = 2.6
  ) +
  coord_cartesian(clip = "off")

p_aligned
```

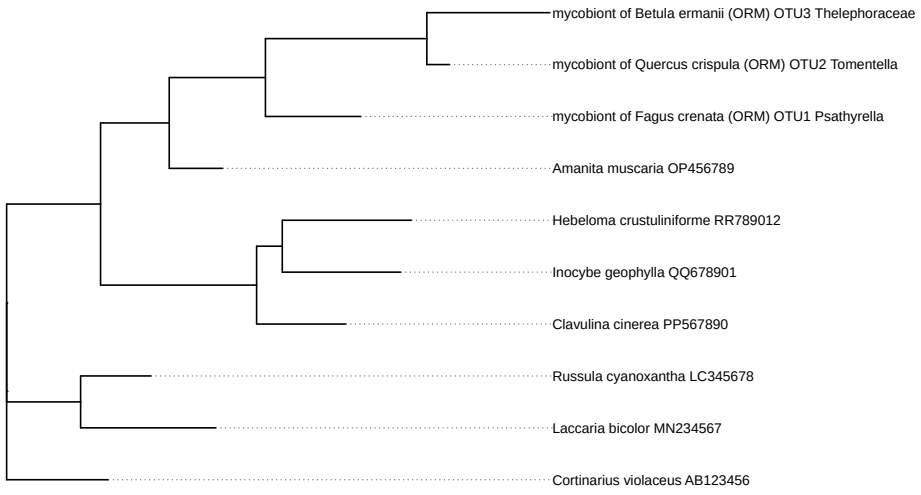


3.6　tip ラベルの見切れを防ぐ（expand = expansion(mult = ...)）

長い学名で tip ラベルが見切れる場合は、x 軸の expand を明示して余白を確保します。必要に応じて coord_cartesian(clip = "off") も併用します。

```
base_tree_tight <- ggtree(tree_pretty)

p_clipping_control <- base_tree_tight +
  geom_segment(
    data = function(df) {
      .tiplab_segment_data(df = df, offset = 0)
    },
    aes(x = x, xend = xend, y = y, yend = yend),
    linetype = "dotted",
    linewidth = 0.35,
```

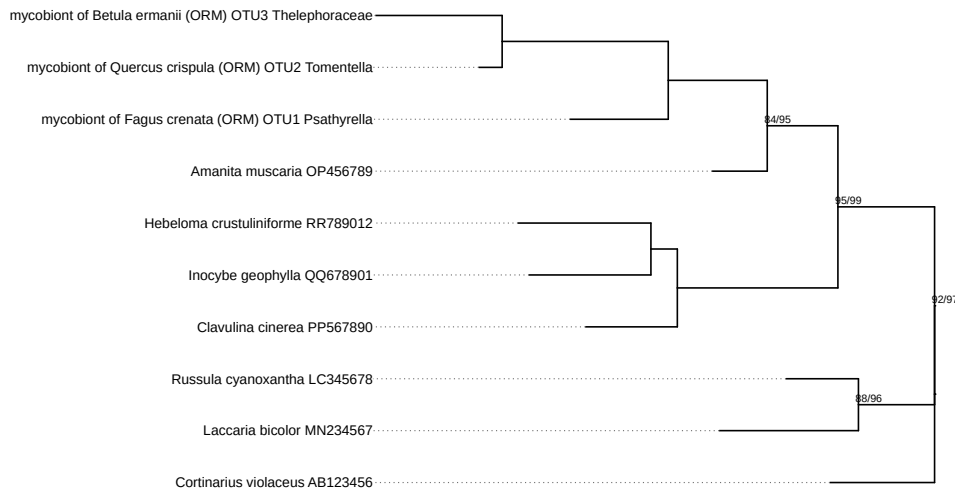
```
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) {
      .tiplab_marquee_data(df = df, offset = 0)
    },
    aes(x = x_lab, y = y, label = label),
    hjust = 0,
    size = 3.2,
    family = "Arial",
    inherit.aes = FALSE
  ) +
  scale_x_continuous(expand = expansion(mult = c(0.02, 0.75))) +
  coord_cartesian(clip = "off")

p_clipping_control
```



3.7 scale_x_reverse() で左右反転する

右側パネル用に系統樹を反転したい場合は、scale_x_reverse() を最後に追加します。反転後は tip ラベルが左側へ出るため、hjust = 1 と expand = expansion(mult = c(..., ...)) の下限側調整が有効です。

```r
p_reversed <- ggtree(tree_pretty) +
  geom_segment(
    data = function(df) {
      .tiplab_segment_data(df = df, offset = 0)
    },
    aes(x = x, xend = xend, y = y, yend = yend),
    linetype = "dotted",
    linewidth = 0.35,
    colour = "grey45",
    inherit.aes = FALSE
  ) +
  geom_marquee(
    data = function(df) {
      .tiplab_marquee_data(df = df, offset = 0)
    },
    aes(x = x_lab, y = y, label = label),
    hjust = 1,
    size = 3.2,
    family = "Arial",
    inherit.aes = FALSE
  ) +
  geom_nodelab(
    aes(label = label),
    hjust = 0,
    vjust = -0.4,
    size = 2.6
  ) +
  coord_cartesian(clip = "off") +
  scale_x_reverse(expand = expansion(mult = c(0.70, 0.02)))

p_reversed
```



3.8 仕様解説

3.8.1 `expand = expansion(mult = c(0.70, 0.02))`

- `expansion(mult = ...)` の `mult` は、データ範囲に対する相対余白です。
- 長さ 2 の場合、`c(a, b)` は軸の両端に別々の余白率を指定します。
- `c(0.70, 0.02)` は「片側を大きく、反対側を小さく」余白確保する設定です。
- `scale_x_reverse()` では表示方向が反転するため、この vignette では ラベル側（左側）に広い余白を取り、反対側を最小限にしています。

3.8.2 `format_node_support()` の仕様

- 入力は node support の文字ベクトル `x`。
- 2 値形式（例: `"SH/UFBoot"`）では、`sh_alrt_cutoff` と `boot_cutoff` でそれぞれ判定します。
- 閾値未満の値は `missing_mark` へ置換され、両方が閾値未満なら空文字 `""` になります。
- 1 値形式は `single_value = "ufboot"` または`"sh_alrt"` で解釈できます。
- `keep_na = TRUE` のときは `NA` を保持し、`sig_digits` で表示桁数を制御します。

簡単な Markdown 解説:

- 文字列そのものを整形する関数で、強調記法は追加しません。
- `missing_mark` の置換結果をそのままラベルとして使えます。

```r
support_example <- c("66.6/59", "90/94", "85/97", "55", "90", NA, "")

tibble(
  input = support_example,
  output_default = format_node_support(support_example),
  output_as_sh = format_node_support(
    support_example,
    single_value = "sh_alrt"
  )
)
```

```
# A tibble: 7 x 3
  input     output_default output_as_sh
  <chr>     <chr>          <chr>
1 "66.6/59" ""             ""
2 "90/94"   "90/—"         "90/—"
3 "85/97"   "85/97"        "85/97"
4 "55"      ""             ""
5 "90"      ""             "90"
6  <NA>      <NA>           <NA>
7 ""        ""             ""
```

3.8.3 `style_tree_label()` の仕様

- 入力は tree label の文字ベクトル `x`。
- 学名に相当する部分は内部で `style_sciname()` を使って整形します。
- 空文字と数値系ラベル（例: `"100"`, `"100/98"`）はそのまま返します。
- `mycobiont_of_...` 形式にも対応し、末尾トークンの扱いは `last_taxon_rank` で補助できます。
- `last_taxon_rank` は `"genus"` / `"species"` などを指定でき、`NA_character_` または `""` で無効化されます。

簡単な Markdown 解説:

- 学名部分に `*...*` が付与され、Markdown italic として描画できます。
- 数値系ラベルは未変換なので、support 表示と混在させやすいです。

```r
tree_label_example <- c(
  "Coprinellus-aureogranulatus_GQ249274.1",
  "mycobiont_of_Cremastra-aphylla_(ORM)_OTU1_Psathyrella",
```

```
  "100/98"
)

tibble(
  input = tree_label_example,
  output_default = style_tree_label(tree_label_example),
  output_genus = style_tree_label(
    tree_label_example,
    last_taxon_rank = "genus"
  )
)
```

```
# A tibble: 3 x 3
  input                                              output_default output_genus
  <chr>                                              <chr>          <chr>
1 Coprinellus-aureogranulatus_GQ249274.1             *Coprinellus*~ *Coprinellu~
2 mycobiont_of_Cremastra-aphylla_(ORM)_OTU1_Psathyr~ mycobiont of ~ mycobiont o~
3 100/98                                             100/98         100/98
```

3.8.4 `style_sciname()` の仕様

- 入力は学名文字列ベクトル `x`。
- 属・種などの学名部分を Markdown italic（`*...*`）へ変換します。
- `cf.`, `aff.`, `nr.` などの同定補助語、`subsp.`, `var.`, `f.` などの階級語を正規化します。
- `×` / `x` の交雑表記にも対応します。
- 数値系ラベル（例: bootstrap 値）は未変換で返します。

簡単な Markdown 解説:

- 学名の主要部分だけに `*...*` を付与し、補助語は通常テキストのまま残します。
- `geom_marquee()` など Markdown 解釈する layer と相性が良いです。

```
sciname_example <- c(
  "Cremastra aphylla",
  "Cremastra cf. appendiculata",
  "×Cremastra appendiculata var. variabilis f. alba",
  "88"
)
```

```r
tibble(
  input = sciname_example,
  output = style_sciname(sciname_example)
)
```

```
# A tibble: 4 x 2
  input                                          output
  <chr>                                          <chr>
1 Cremastra aphylla                              *Cremastra* *aphylla*
2 Cremastra cf. appendiculata                    *Cremastra* cf. *appendicula
3 ×Cremastra appendiculata var. variabilis f. alba ×*Cremastra* *appendiculata*
4 88                                             88
```

3.9　実務メモ

- 系統推定時に (ORM) のような括弧付き表記がプログラム側で_ORM_ へ置換される場合があります。style_tree_label() の前に str_replace_all() で_(ORM)_ 形式へ戻しておくと整形が安定します。
- support 値は format_node_support() で閾値を固定しておくと、図間比較時の解釈がぶれにくくなります。
- offset を小さく調整すると、狭い図幅でもラベル同士の衝突を減らせます。
- 見切れ対策は scale_x_*() の expand = expansion(mult = c(..., ...)) を先に調整し、plot.margin には依存しない設定にしておくと再利用しやすくなります。
- 反転パネルでは scale_x_reverse(expand = ...) と hjust = 1 をセットで調整すると、ラベルの位置合わせが安定します。

第 4 章

Session info

再現性のため、本書をレンダリングした実行環境情報を記録します。

```r
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.4

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK versio

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Asia/Tokyo
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

loaded via a namespace (and not attached):
 [1] compiler_4.5.1    fastmap_1.2.0     cli_3.6.5         tools_4.5.1
 [5] htmltools_0.5.8.1 rstudioapi_0.17.1 yaml_2.3.10       rmarkdown_2.29
 [9] knitr_1.50        jsonlite_2.0.0    xfun_0.53         digest_0.6.37
[13] rlang_1.1.6       evaluate_1.0.4
```